YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

Project – Low Level Design

On

Predictive Modeling for Retail Success

SUBMITTED BY

Razal Rafeeque Kottat-23BCDSBA26

BCA(Data Science and Big Data Analytics)

UNDER THE GUIDANCE OF

Industry Mentor:Mr.Ujjwal Kumar

Guided by:Ms.Raksha Adyanthaya

TABLE OF CONTENTS

1. Introduction.

1.1. Scope of the document.

1.2. Intended Audience.

1.3. System overview.

2. System Design.

2.1. Application Design.

2.2. Process Flow.

2.3. Information Flow.

2.4. Components Design

2.5. Key Design Considerations.

2.6. API Catalogue.

3. Data Design.

3.1. Data Model.

3.2. Data Access Mechanism.

3.3. Data Retention Policies.

3.4. Data Migration.

TABLE OF CONTENTS

4. Interfaces.

5. State and Session Management.

6. Caching.

7. Non-Functional Requirements.

7.1. Security Aspects.

7.2. Performance Aspects.

8. References.

1. INTRODUCTION

This document explains the internal working of the Customer Response Prediction System. It describes modules, data processing, and prediction logic.

1.1 . Scope of the document.

This document provides a detailed low-level design of the Customer Response Prediction System. It explains the internal working, architecture, data flow, and implementation details required to develop a machine learning model that predicts whether a customer will respond to a marketing campaign.

The scope includes:

. Data preprocessing
. Feature engineering
. Model training and validation
. Prediction and evaluation

1.2. Intended Audience.

This document is intended for:
. Software Developers
. Data Scientists
. Project Evaluators
. Academic Reviewers

1.3. System overview.

The system uses customer data such as income, age, spending behavior, and purchase history to predict whether a customer will respond to a marketing campaign. The output is binary:
. 0 → No Response
. 1 → Response
The system uses machine learning (XGBoost) to improve prediction accuracy.

2.SYSTEM DESIGN

2.1. Application Design.

The application follows a modular architecture:

. Data Collection Module
. Data Preprocessing Module
. Feature Engineering Module
. Model Training Module
. Evaluation Module
. Prediction Module

This modular approach ensures scalability and easy maintenance.

2.2. Process Flow.

1. Load dataset from CSV file
2. Clean and preprocess data
3. Perform feature engineering
4. Visualize data using graphs
5. Split dataset into training and testing
6. Apply cross-validation
7. Train model using XGBoost
8. Evaluate model performance
9. Predict customer response

2.3. Information Flow.

. Input: Customer dataset
. Processing: Cleaning → Feature Engineering → Model Training
. Output: Prediction (0 or 1)

2.4. Components Design.

Data Module

. Handles data loading and preprocessing

Feature Engineering Module

Creates new features like:

. Total Spending

. Total Purchases

. Age

Model Module

Uses XGBoost for prediction

Evaluation Module

. Accuracy

. Confusion Matrix

. ROC Curve

Prediction Module

. Predicts response for new/unseen customers

2.5. Key Design Considerations.

. Accuracy of prediction

. Simplicity and interpretability

. Efficient data handling

. Scalability for larger datasets

2.6. API Catalogue.

(Not implemented – optional for future work)

Example (future):

. /predict → returns prediction (0/1)

. /train → retrains model

3. DATA DESIGN.

3.1. Data Model.

Feature	Description
Income	Customer income
Age	Customer age
Total_Spending	Total spending
Total_Purchases	Number of purchases
Response	Target variable (0/1)

3.2. Data Access Mechanism.

. Data is loaded using pandas:

pd.read_csv("superstore_data.csv")

3.3. Data Retention Policies.

. Data is stored locally
. No long-term storage or sensitive data handling

3.4. Data Migration.

Not applicable (single dataset used)

4. Interfaces.

. User interacts through Jupyter Notebook
. Inputs: Dataset / custom customer data
. Outputs: Predictions and graphs

5. State and Session Management.

. No session management required
. Each run is independent

6. Caching.

. No caching implemented
. Data processed in real-time

7. Non-Functional Requirements.

7.1 Security Aspects.

. No sensitive data used
. Local execution ensures data privacy

7.2 Performance Aspects

. Fast execution for small dataset
. XGBoost ensures efficient training
. Cross-validation improves reliability

8. References

. Scikit-learn documentation
. XGBoost documentation
. Python (Pandas, NumPy, Matplotlib) libraries